Dictionnaire des données — toutes les bases

Hôtel & Restaurant — Dictionnaire des données V1 (MPD Merise)

Ce document explique **chaque table et chaque attribut** des 7 bases de données : ce qu'il contient, **pourquoi** il existe et **quelle logique** l'utilise. Il correspond au **MPD** (modèle physique) de Merise : les fichiers sources sont `services/<service>/prisma/schema.prisma` et les migrations SQL dans `services/<service>/prisma/migrations/`.

---

0. Les conventions (à lire en premier, elles valent pour toutes les tables)

Convention	Exemple	Pourquoi
Clé primaire `id` en UUID	`78072c82-869c-…`	Généré sans demander à la base, impossible à deviner (on ne peut pas essayer `id=1, 2, 3…` pour lire les données des autres), et unique **entre services** : un `reservationId` créé par hebergement peut voyager dans un événement sans collision.
Nom en camelCase dans le code, snake_case en base	`tarifNuit` ↔ `tarif_nuit` (`@map`)	Convention de TypeScript d'un côté, de PostgreSQL de l'autre. `@@map("chambre")` donne le nom de la table.
Montants en `Int` (entier), en Ariary	`85000`	L'Ariary n'a pas de centimes en pratique, et un entier n'a **jamais d'erreur d'arrondi** (contrairement à `0.1 + 0.2` en virgule flottante).
`createdAt` / `updatedAt`	`@default(now())` / `@updatedAt`	Traçabilité : quand la ligne a été créée, quand elle a changé pour la dernière fois. Prisma remplit `updatedAt` tout seul.
`Timestamptz` pour un instant, **`Date`** pour un jour	`checkInLe` vs `dateArrivee`	Un instant (« 14 h 32 à Tana ») a besoin du fuseau horaire ; une date de séjour (« le 15 octobre ») n'a pas d'heure. Mélanger les deux crée des décalages d'un jour.
`Json` pour les textes traduits (`TexteTraduit`)	`{"fr": "Chambre standard", "mg": "Efitra tsotra"}`	Une seule colonne pour toutes les langues. Ajouter l'anglais = ajouter une clé `en`, sans migration.
« sans FK » + « snapshot »	`clientId` (sans FK) + `clientNom` (snapshot)	**Règle de la SOA** : une base n'a pas de clé étrangère vers la base d'un autre service (ce sont des bases séparées). On garde l'**identifiant** (pour retrouver la donnée) et une **copie figée** des champs utiles (pour afficher sans appeler l'autre service, et pour garder la valeur **au moment de l'action** : si le client change de nom, sa facture garde l'ancien).
`version`	`version Int @default(1)`	**Verrou optimiste** (commande, reservation, tache_menage) : chaque modification fait `version + 1` et exige la version lue (`where: { id, version }`). Si deux personnes modifient en même temps, la seconde modifie 0 ligne et reçoit un conflit (409) au lieu d'écraser la première (« mise à jour perdue »). Pour `chambre`, l'incrément sert à **verrouiller la ligne** pendant le check-in (voir cette table).
Enum	`statut StatutReservation`	Liste fermée de valeurs, vérifiée par la base : une faute de frappe (`"CONFIMEE"`) est impossible.

Convention	Exemple	Pourquoi
Contraintes `CHECK` / `EXCLUDE` dans les migrations	`CHECK (quantite >= 0)`	Prisma ne sait pas les écrire dans `schema.prisma` : elles sont ajoutées **à la main** dans le SQL de migration. C'est la **dernière ligne de défense** : même un bug dans le code ne peut pas les violer.

Les 3 tables techniques (présentes dans plusieurs services)

`outbox_evenement` — patron *Transactional Outbox* (hebergement, facturation, menage, stock, restaurant, comptabilite)

Attribut	Type	Description et logique
`id`	uuid	Sert aussi d'**identifiant de l'événement** (les consommateurs s'en servent pour l'idempotence).
`type`	texte	Nom de l'événement : `reservation.creee`, `menage.termine`… (liste dans `packages/commun/src/evenements.ts`).
`payload`	json	Le contenu de l'événement, validé par un schéma Zod.
`createdAt`	instant	Ordre de publication.
`publieLe`	instant, **nul tant que non publié**	Le relais lit les lignes où `publieLe` est nul, les envoie dans Redis/BullMQ, puis remplit la date. Index `(publieLe, createdAt)` pour trouver vite les événements en attente.

Pourquoi : l'événement est écrit dans **la même transaction** que la donnée métier. Soit les deux sont enregistrés, soit aucun. Si Redis est en panne, l'événement attend ici et part plus tard : il n'est jamais perdu.

`evenement_traite` — consommateur idempotent

Attribut	Type	Description et logique
`evenementId`	uuid, **clé primaire**	L'id de l'événement reçu. Inséré **avant** de traiter, dans la même transaction que l'effet. Si le même événement arrive deux fois, l'insertion viole la clé primaire, la transaction est annulée : **aucun double effet** (jamais deux tâches de ménage pour le même séjour). Utilisé par la fonction `traiterUneFois` (`src/outbox.ts` de chaque service). Exception : stock s'appuie sur la clé unique métier `(referenceId, articleId)` de `mouvement`, une transaction par article.
`traiteLe`	instant	Quand il a été traité.

`compteur_<x>` (`compteur_reservation`, `compteur_facture`, `compteur_commande`)

Attribut	Type	Description et logique
`annee`	entier, **clé primaire**	Un compteur par année : la numérotation repart à 1 chaque 1er janvier.
`dernierNumero`	entier	Dernier numéro donné. Un `upsert` + `increment` **atomique** garantit que deux créations simultanées n'obtiennent jamais le même numéro → `RES-2026-000042`, `FAC-2026-000123`, `CMD-2026-000045`. Pour les factures, la numérotation **sans trou** est une obligation comptable.

1. Base `auth_db` — service auth (comptes et connexion)

Table `utilisateur` — toute personne qui se connecte (personnel et clients)

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant. Il est mis dans le **JWT** (`sub`) et recopié dans les autres services comme `clientId`, `assigneeId`, `encaissePar`…
`nom`	texte	oui	Nom affiché (« Soa Randriamanana »).
`telephone`	texte, **unique**	oui	**Identifiant de connexion principal** (à Madagascar, tout le monde a un téléphone, pas toujours un email). Normalisé `0341234567` pour que `034 12 345 67` et `+261341234567` désignent le même compte.
`email`	texte, **unique**	oui	Second identifiant de connexion ; reçus et notifications. Toujours en minuscules (sinon `Jean@mail.mg` et `jean@mail.mg` seraient deux comptes).
`motDePasseHash`	texte	oui	**Jamais le mot de passe** : son empreinte **scrypt** (`scrypt$N$r$p$sel$empreinte`). scrypt est volontairement lent et coûteux en mémoire, ce qui rend le cassage par force brute très cher. Le **sel** (aléatoire, différent par compte) empêche deux mots de passe identiques d'avoir la même empreinte.
`role`	enum `Role`	oui	`ADMIN`, `RECEPTIONNISTE`, `FEMME_MENAGE`, `RESTAURANT`, `COMPTABLE`, `COMMUNICATION`, `CLIENT`. Mis dans le JWT ; la gateway s'en sert avec sa table `REGLES` pour autoriser ou refuser chaque route.
`langue`	enum `Langue`	oui (défaut `FR`)	`FR`, `MG`, `EN` : langue des textes et des notifications de cet utilisateur.
`actif`	booléen	oui (défaut vrai)	Un compte **désactivé** ne peut plus se connecter ni rafraîchir sa session. On **désactive** au lieu de supprimer : les anciennes actions (paiements encaissés, ménages faits) gardent leur auteur.
`createdAt`, `updatedAt`	instant	oui	Traçabilité.

Table `session` — une connexion ouverte (jeton de rafraîchissement)

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`utilisateurId`	uuid, **FK → utilisateur** (suppression en cascade)	oui	À qui appartient la session (même base : la FK est permise).
`jetonHash`	texte, **unique**	oui	**Empreinte SHA-256** du jeton de rafraîchissement (le jeton en clair est dans le cookie du navigateur, pas en base). Si la base fuit, les jetons ne sont pas utilisables.
`expireLe`	instant	oui	Fin de validité. Après, il faut se reconnecter.
`revoqueLe`	instant	non	Rempli à la déconnexion, ou à la **rotation** : à chaque rafraîchissement, l'ancien jeton est révoqué et un nouveau est créé. Un jeton volé et réutilisé est donc détecté.
`createdAt`	instant	oui	Date de connexion.

Logique de connexion : le JWT d'accès (court, signé par la clé privée d'auth, vérifié par la gateway avec la clé publique) + la session (longue, révocable). Index sur `utilisateurId` pour révoquer vite toutes les sessions d'un compte désactivé.

---

2. Base `hebergement_db` — service hebergement (chambres, réservations, séjours)

Table `categorie_chambre` — un type de chambre vendu

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`code`	enum `CodeCategorie`, **unique**	oui	`STANDARD`, `SUITE_SENIOR`, `SUITE_PRESTIGE`. Code stable, utilisable dans les URL et les traductions (le libellé, lui, peut changer).
`libelle`	json (traduit)	oui	Nom affiché : `{"fr": "Suite senior", "mg": "…"}`.
`description`	json (traduit)	non	Texte de présentation.
`capacite`	entier, **CHECK > 0**	oui	Nombre maximal de personnes.
`tarifNuit`	entier (Ar), **CHECK > 0**	oui	Prix **par défaut** d'une nuit, utilisé hors saison.

Le client réserve une catégorie, pas une chambre : c'est l'hôtel qui choisit la chambre libre (la plus simple à gérer pour lui).

Table `chambre` — une chambre physique

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`numero`	texte, **unique**	oui	« 204 ». Texte et non nombre : certains hôtels ont « 12B ».
`etage`	entier	oui	Pour le tableau des chambres de la réception.
`categorieId`	uuid, **FK → categorie_chambre** (suppression interdite)	oui	Type de la chambre. On ne peut pas supprimer une catégorie qui a encore des chambres. Index pour trouver vite les chambres d'une catégorie.
`etatMenage`	enum `EtatMenage`	oui (défaut `PROPRE`)	`PROPRE` → `SALE` (au check-out) → `EN_NETTOYAGE` (ménage démarré) → `PROPRE` (événement `menage.termine`). **Le check-in exige `PROPRE`.**
`horsService`	booléen	oui (défaut faux)	Chambre en travaux : jamais attribuée à une réservation.
`version`	entier	oui	**Écrire pour verrouiller** : au check-in, `updateMany({ where: { id, etatMenage: PROPRE, horsService: false }, data: { version: +1 } })` vérifie que la chambre est propre **et** verrouille sa ligne jusqu'au commit, en une seule instruction. Personne ne peut la passer à `SALE` entre la vérification et l'installation du client. (Deux réservations de la même chambre aux mêmes dates, elles, sont empêchées par la contrainte `EXCLUDE` de `reservation`.)

Tables `saison` et `tarif_saison` — prix différents selon la période

`saison`

Attribut	Type	Description et logique
`id`	uuid	Identifiant.
`nom`	json (traduit)	« Haute saison », « Fêtes de fin d'année ».
`dateDebut`, `dateFin`	date	Période **incluse** (`[]`). Contrainte `EXCLUDE saison_sans_chevauchement` : deux saisons ne peuvent pas se chevaucher, sinon une nuit aurait deux prix.

`tarif_saison` — table **d'association** (relation plusieurs-à-plusieurs entre saison et catégorie, avec un attribut porté)

Attribut	Type	Description et logique
`saisonId`	uuid, FK → saison (cascade)	
`categorieId`	uuid, FK → categorie_chambre	
`tarifNuit`	entier (Ar)	Le prix d'une nuit de **cette** catégorie pendant **cette** saison.
Clé primaire	`(saisonId, categorieId)`	Clé **composée** : un seul prix par couple saison × catégorie. C'est exactement l'**association porteuse de données** du MCD Merise.

Table `reservation` — une réservation, puis un séjour

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant technique.
`reference`	texte, **unique**	oui	`RES-2026-000042` : lisible, à donner au téléphone (vient de `compteur_reservation`).
`chambreId`	uuid, FK → chambre	oui	La chambre attribuée par l'hôtel (peut changer au check-in si elle n'est pas propre).
`clientId`	uuid (auth, **sans FK**)	**non**	Le compte du client s'il a réservé en ligne ; **nul** pour une réservation au comptoir d'un client sans compte. Index pour « Mes réservations ».
`clientNom`, `clientTelephone`, `clientEmail`	texte (**snapshots**)	oui	Coordonnées **au moment de la réservation** ; présentes même sans compte. L'email est obligatoire : le reçu part par email.
`pieceIdentiteType`, `pieceIdentiteNumero`	texte	oui	CIN ou passeport + numéro : exigé par la réglementation hôtelière, vérifié au check-in. **Donnée personnelle sensible.**
`dateArrivee`, `dateDepart`	date	oui	Le séjour. **CHECK** `date_depart > date_arrivee` (au moins une nuit).
`montantTotal`	entier (Ar), CHECK ≥ 0	oui	**Calculé par le serveur** (somme des nuitées), jamais envoyé par le navigateur.

Attribut	Type	Obligatoire	Description et logique
`statut`	enum `StatutReservation`	oui	Machine à états : `EN_ATTENTE_PAIEMENT` → `CONFIRMEE` (paiement reçu) → `EN_COURS` (check-in) → `TERMINEE` (check-out) ; ou `ANNULEE` (non payée à temps, ou par la réception). Index pour filtrer par statut.
`canal`	enum `CanalReservation`	oui	`SITE_WEB`, `RECEPTION`, `FACEBOOK`, `ASSISTANT_IA` (prévu V2). Sert aux statistiques « nuitées par canal » de la comptabilité.
`langue`	enum	oui	Langue des messages envoyés pour cette réservation.
`expireLe`	instant	non	Fin des **30 minutes** pour payer. Un job BullMQ différé l'annule à cette heure s'il est toujours `EN_ATTENTE_PAIEMENT`. Remis à nul une fois confirmée ou annulée.
`checkInLe`, `checkOutLe`	instant	non	Heures réelles d'arrivée et de départ.
`version`	entier	oui	Verrou optimiste : par exemple, l'expiration et la confirmation de paiement arrivent en même temps ; une seule gagne.

La règle la plus importante de la base — contrainte `EXCLUDE` `reservation_sans_chevauchement` :

```
EXCLUDE USING gist ("chambre_id" WITH =, daterange("date_arrivee", "date_depart", '[)') WITH &&)
WHERE ("statut" IN ('EN_ATTENTE_PAIEMENT', 'CONFIRMEE', 'EN_COURS'))
```

Traduction : « deux réservations **actives** de la **même chambre** (`=`) ne peuvent pas avoir des périodes qui **se chevauchent** (`&&`) ». `'[)'` exclut la date de départ : un départ le 13 et une arrivée le 13 sont permis. Les réservations annulées ou terminées ne comptent pas. C'est **PostgreSQL** qui empêche le surbooking, même avec deux clics à la même milliseconde. (`btree_gist` est l'extension qui permet de combiner `=` et `&&` dans une même contrainte.)

Index `(chambreId, dateArrivee)` : pour chercher vite une chambre libre.

Table `reservation_nuitee` — le détail nuit par nuit

Attribut	Type	Description et logique
`reservationId`	uuid, FK → reservation (cascade)	
`nuit`	date	Une ligne par nuit du séjour.
`tarifApplique`	entier (Ar)	Le prix **réellement appliqué** cette nuit-là (tarif de saison ou tarif par défaut), **figé** : si l'admin change les tarifs demain, la réservation ne change pas.
`saisonId`	uuid, FK → saison (mise à nul si la saison est supprimée)	Quelle saison a fixé ce prix (nul = tarif par défaut).
Clé primaire	`(reservationId, nuit)`	Une seule ligne par nuit.

`montantTotal` de la réservation = somme des `tarifApplique`. C'est cette table qui servira à afficher le « détail des nuitées » (F5 du backlog).

Table `cle_idempotence` — rejouer une requête sans créer de doublon

Attribut	Type	Description et logique
`cle`	uuid, **clé primaire**	La valeur de l'en-tête `Idempotency-Key` envoyé par le front.
`route`	texte	Sur quelle route elle a servi.
`utilisateurId`	uuid	L'auteur : sinon, quelqu'un qui devinerait la clé pourrait relire la réponse d'un autre client.
`reponse`	json	La réponse déjà renvoyée.
`createdAt`	instant	Purge après 7 jours.

Logique : le client clique « Réserver », le réseau coupe, le navigateur renvoie la requête avec la **même clé** → la base renvoie la réponse déjà enregistrée au lieu de créer une deuxième réservation.

---

3. Base `facturation_db` — service facturation (factures et paiements)

Table `facture`

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`numero`	texte, **unique**	oui	`FAC-2026-000123`, séquentiel **sans trou** (`compteur_facture`, même transaction) : obligation comptable.
`type`	enum `TypeFacture`	oui	`SEJOUR` (une réservation) ou `RESTAURANT` (une commande payée sur place).
`reservationId`	uuid, **unique**, sans FK	non	Pour une facture de séjour. **Unique** : une réservation n'a qu'**une** facture, même si l'événement `reservation.creee` arrive deux fois.
`commandeId`	uuid, **unique**, sans FK	non	Pour une facture restaurant.
`clientId`	uuid (auth), sans FK	non	Le compte du client : il ne voit et ne paie **que ses** factures.
`clientNom`, `clientEmail`, `clientTelephone`	texte (**snapshots**)	nom oui	Pour le reçu et pour pré-remplir le paiement mobile.
`langue`	enum	oui	Langue de la facture.
`statut`	enum `StatutFacture`	oui	`EMISE` → `PAYEE` (montant atteint) → `CLOTUREE` (séjour terminé, copie envoyée à la comptabilité) ; ou `ANNULEE` (réservation expirée). **Une facture émise ne se supprime jamais** (traçabilité comptable).
`montantTotal`	entier (Ar)	oui	Somme des lignes. Augmente si un repas est imputé à la chambre.

Attribut	Type	Obligatoire	Description et logique
`montantPaye`	entier (Ar)	oui	Somme des paiements réussis. **CHECK `facture_pas_de_surpaiement` : `montant_paye <= montant_total`** — la base refuse d'encaisser plus que dû (ex. espèces à la réception + Papi en même temps).
`emiseLe`, `clotureeLe`	instant	| Dates comptables.	

Reste à payer = `montantTotal - montantPaye` (calculé, jamais stocké : une valeur calculée stockée finit toujours par être fausse).

Table `ligne_facture`

Attribut	Type	Description et logique
`id`	uuid	
`factureId`	uuid, FK → facture (cascade)	
`libelle`	texte	« Suite Prestige – 3 nuits », « Restaurant – CMD-2026-000045 ».
`quantite`, `prixUnitaire`, `montant`	entiers, CHECK > 0 / ≥ 0	`montant = quantite × prixUnitaire`.
`origine`	enum `Origine`	**À qui appartient l'argent** : `HEBERGEMENT`, `RESTAURANT`, `LIVRAISON` (V3). Un repas imputé à la chambre est **payé à l'hôtel** mais **appartient au restaurant** → la comptabilité crée un reversement.
`sourceId`	uuid, sans FK	L'id de la réservation ou de la commande facturée.
Unique	`(origine, sourceId)`	Une commande n'est facturée **qu'une fois**, même si `consommation.imputee` est rejoué.

Table `paiement`

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Envoyé à Papi comme **`reference`** ; Papi le renvoie dans `merchantPaymentReference` → vérification de cohérence.
`factureId`	uuid, FK → facture	oui	La facture payée (plusieurs paiements possibles : un acompte Papi + le reste en espèces).
`montant`	entier, CHECK > 0	oui	**Le reste à payer calculé par le serveur** au moment de la demande. Le montant notifié par Papi doit être **exactement** celui-ci, sinon `ANOMALIE`.
`statut`	enum `StatutPaiement`	oui	`EN_ATTENTE` (lien créé) → `REUSSI` / `ECHOUE` ; `ANOMALIE` = argent reçu mais incohérent (mauvais montant, facture déjà soldée, lien remplacé) → **remboursement manuel**.
`methode`	enum `MethodePaiement`	non	`ESPECES`, `MVOLA`, `ORANGE_MONEY`, `AIRTEL_MONEY`, `CARTE`. Inconnue tant que Papi n'a pas dit comment le client a payé.
`referenceExterne`	texte, **unique**	non	`paymentReference` de Papi : une même transaction Papi ne peut être enregistrée **qu'une fois**.

Attribut	Type	Obligatoire	Description et logique
`encaissePar`	uuid (auth), sans FK	non	Pour les espèces : qui a encaissé (contrôle de caisse).
`jetonHash`	texte	non	**Empreinte SHA-256** du jeton secret placé dans l'URL de notification. **1re vérification** d'une notification : elle doit présenter le bon jeton. On ne garde que l'empreinte : même avec une copie de la base, impossible de fabriquer une notification valide.
`notificationTokenHash`	texte	non	Empreinte du `notificationToken` que Papi a renvoyé à la création du lien. **2e vérification**, faite par le worker.
`urlPaiement`	texte	non	L'adresse de la page Papi, pour la renvoyer en cas de double clic (règle corrigée par **B2**).
`erreur`	texte	non	Pourquoi le paiement a échoué (« Solde insuffisant », « Remplacé par un nouveau lien »…).

Index unique partiel `paiement_un_en_attente_par_facture` (SQL écrit à la main) :

```
CREATE UNIQUE INDEX … ON paiement (facture_id) WHERE statut = 'EN_ATTENTE';
```

→ **au plus un** paiement en attente par facture : trois clics simultanés ne créent pas trois liens Papi.

---

4. Base `menage_db` — service menage

Table `tache_menage`

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	
`reservationId`	uuid (hebergement), sans FK	oui	Le séjour concerné.
`chambreId`	uuid (hebergement), sans FK	oui	Renvoyé dans `menage.termine` pour que hebergement remette la chambre `PROPRE`.
`chambreNumero`	texte (**snapshot**)	oui	Pour afficher « Chambre 204 » sans appeler hebergement.
`type`	enum `TypeTache`	oui	`QUOTIDIEN` (chaque jour du séjour, créé par `sejour.commence`) ou `DEPART` (au check-out, créé par `sejour.termine`, affiché en premier : la chambre doit être prête pour le client suivant).
`datePrevue`	date	oui	Le jour où la tâche doit être faite.
`statut`	enum `StatutTache`	oui	`A_FAIRE` → `EN_COURS` (Démarrer) → `TERMINEE`.
`assigneeId`	uuid (auth), sans FK	non	La femme de ménage qui a **démarré** la tâche. **Seule elle** peut la terminer.
`assigneeNom`	texte (**snapshot**)	non	« Hanta Rasoanirina ».

Attribut	Type	Obligatoire	Description et logique
`commenceLe`, `termineLe`	instant	non	**CHECK : fin ≥ début.** Mesurent la durée du ménage.
`remarques`	texte	non	« Ampoule à changer », objet oublié.
`version`	entier	oui	Verrou optimiste : deux collègues cliquent « Démarrer » en même temps, une seule l'obtient.

Unique `(reservationId, type, datePrevue)` : un événement rejoué ne crée jamais deux fois la même tâche. **Index** `(datePrevue, statut)` : l'écran « Tâches du jour » filtre exactement là-dessus. **À venir (F8)** : `debutAuPlusTot` / `finAuPlusTard` pour les heures du ménage.

---

5. Base `stock_db` — service stock (articles d'accueil)

Table `article`

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	
`code`	texte, **unique**	oui	`GEL_DOUCHE`, `PAPIER_HYGIENIQUE`, `PANTOUFLE`, `BROSSE_A_DENTS` : le ménage déclare ses articles par code.
`libelle`	json (traduit)	oui	
`quantite`	entier, **CHECK `stock_jamais_negatif` ≥ 0**	oui	Stock **courant**, pour une lecture rapide. Décrémenté **atomiquement** (`quantite = quantite - n`) : deux ménages simultanés ne peuvent pas faire passer le stock sous zéro.
`seuilAlerte`	entier, CHECK ≥ 0	oui	En dessous → « Stock bas » (en rouge) et événement `stock.seuil_atteint`.
`consommationParNuitee`	entier, CHECK ≥ 0	oui (défaut 1)	Combien on en consomme par nuit vendue. Sert au **contrôle** de la comptabilité : consommation attendue = nuitées × ce nombre.

Table `mouvement` — l'historique (la preuve)

Attribut	Type	Description et logique
`id`	uuid	
`articleId`	uuid, FK → article	
`type`	enum `TypeMouvement`	`ENTREE` ou `SORTIE` : **le sens**.
`motif`	enum `MotifMouvement`	`APPROVISIONNEMENT`, `MENAGE`, `AJUSTEMENT_INVENTAIRE`, `PERTE` : **la raison**. Index `(motif, createdAt)` pour « sorties MENAGE du jour » (contrôle comptable).
`quantite`	entier, **CHECK > 0**	Toujours positive : le sens vient de `type` (on évite les quantités négatives ambiguës).

Attribut	Type	Description et logique
`referenceId`	uuid, sans FK	La tâche de ménage à l'origine de la sortie.
`chambreNumero`	texte (snapshot)	« Sortie : ménage chambre 204 ».
`effectuePar`	uuid (auth), sans FK	Qui a fait le mouvement manuel.
Unique	`(referenceId, articleId)`	Une tâche de ménage ne sort **qu'une fois** chaque article (idempotence de `menage.termine`).

Logique : `article.quantite` et `mouvement` sont modifiés dans **la même transaction**. La quantité dit « combien maintenant », l'historique dit « pourquoi ».

---

6. Base `restaurant_db` — service restaurant

Table `menu`

Attribut	Type	Description et logique
`id`	uuid	
`nom`	json (traduit)	« Carte malgache ».
`actif`	booléen (défaut faux)	**Publié** ou non. Seuls les menus actifs apparaissent sur le site (c'est la Communication qui publie).

Table `plat`

Attribut	Type	Description et logique
`id`	uuid	
`menuId`	uuid, FK → menu	Le menu qui contient le plat.
`nom`, `description`	json (traduit)	
`categorie`	enum `CategoriePlat`	`ENTREE`, `PLAT`, `DESSERT`, `BOISSON` : l'ordre d'affichage de la carte.
`prix`	entier (Ar), **CHECK > 0**	**Le seul prix qui compte** : le serveur le relit à chaque commande, le navigateur n'envoie que l'id du plat et la quantité.
`imageUrl`	texte	Prévu, pas encore utilisé en V1.
`tempsPreparationMin`	entier (défaut 15)	Pour estimer l'attente.
`disponible`	booléen	Faux = « Épuisé » : disparaît de la prise de commande, barré sur le site.
`disponibleADistance`	booléen	Prévu pour la commande à distance (E5), pas utilisé en V1.

À venir (F7 / E2) : tables `ingredient` et `recette_plat` (plat × ingrédient × quantité).

Table `commande`

Attribut	Type	Description et logique
`id`	uuid	

Attribut	Type	Description et logique
`reference`	texte, **unique**	`CMD-2026-000045` (`compteur_commande`).
`canal`	enum `CanalCommande`	`SUR_PLACE`, `A_EMPORTER`, `LIVRAISON` (prévu).
`tableNumero`	texte	Pour le service en salle.
`clientId`	uuid (auth), sans FK	Si un client connecté commande (prévu E5).
`clientNom`, `clientTelephone`	texte (snapshots)	
`heureSouhaitee`	instant	Prévu pour la commande à distance.
`statut`	enum `StatutCommande`	**Préparation** : `RECUE` → `EN_PREPARATION` → `PRETE` → `REMISE`, ou `ANNULEE`.
`statutPaiement`	enum `StatutPaiementCommande`	**Paiement**, indépendant de la préparation : `NON_PAYEE`, `PAYEE`, `IMPUTEE_CHAMBRE` (ajoutée à la facture du séjour), `A_LA_LIVRAISON` (prévu). Deux statuts séparés car on peut servir avant de payer, ou payer avant d'être servi.
`reservationId`, `chambreNumero`	uuid sans FK + snapshot	Remplis à l'**imputation à une chambre** : le séjour en cours dans cette chambre. Déclenche l'événement `consommation.imputee` vers facturation.
`montantTotal`	entier, CHECK ≥ 0	Somme des lignes, calculée par le serveur.
`fraisLivraison`	entier, CHECK ≥ 0	Prévu (V3).
`serviPar`	uuid (auth), sans FK	Le serveur qui a pris la commande.
`version`	entier	Verrou optimiste (deux serveurs changent le statut en même temps).

Table `ligne_commande`

Attribut	Type	Description et logique
`id`	uuid	
`commandeId`	uuid, FK → commande (cascade)	
`platId`	uuid, FK → plat	
`platNom`	texte (**snapshot**)	Le nom au moment de la commande, dans la langue de la commande.
`prixUnitaire`	entier (**snapshot**), CHECK > 0	**Un changement de prix demain ne modifie pas les commandes passées.**
`quantite`	entier, CHECK > 0	

---

7. Base `comptabilite_db` — service comptabilite (modèle de lecture)

Ce service **ne crée aucune donnée métier** : il **reçoit des copies** (événements) et **calcule**. Tout peut être recalculé. C'est un **modèle de lecture** (idée du patron CQRS) : des données organisées pour être **lues** vite par le comptable, sans interroger les autres services.

Table `copie_facture` (reçue par `facture.cloturee`)

Attribut	Type	Description et logique
`factureId`	uuid, **clé primaire**, sans FK	Une seule copie par facture, même si l'événement est rejoué.
`numero`, `type`, `clientNom`	texte	Copie figée.
`montantTotal`, `montantHebergement`, `montantRestaurant`	entiers	Totaux **déjà ventilés** par origine : le chiffre d'affaires du jour se calcule sans relire les lignes.
`clotureeLe`	instant, index	« Factures du jour ».
`lignes`	json	Le détail, pour l'écran « Copies de factures ».
`recueLe`	instant	Quand la copie est arrivée.

Table `statistique_journaliere` (générée à 23 h ou à la demande)

Attribut	Type	Description et logique
`jour`	date, **clé primaire**	Un tableau par jour (le régénérer remplace l'ancien).
`nuiteesVendues`	entier	
`nombreFactures`	entier	Factures clôturées ce jour.
`caHebergement`, `caRestaurant`, `caLivraison`	entiers (Ar)	Chiffre d'affaires par activité (`caLivraison` : V3).
`repartitionCanaux`	json	`{"SITE_WEB": 4, "RECEPTION": 7, "FACEBOOK": 2}`. Json car la liste des canaux peut évoluer.
`genereLe`	instant	

Table `controle_stock_journalier`

Attribut	Type	Description et logique
`jour` + `articleCode`	**clé primaire composée**	Une ligne par article et par jour.
`consommationAttendue`	entier	Nuitées vendues × `consommation_par_nuitee`.
`consommationReelle`	entier	Sorties `MENAGE` du jour (déclarées par le ménage).
`ecart`	entier	Réelle − attendue. **≠ 0 → à signaler à la direction** (vol, oubli de déclaration, erreur).

Table `reversement`

Attribut	Type	Description et logique
`id`	uuid	
`factureId`	uuid, **unique**, sans FK	Un reversement par facture au plus.
`numeroFacture`	texte (snapshot)	

Attribut	Type	Description et logique
`montant`	entier, **CHECK > 0**	Total des lignes d'origine `RESTAURANT` de la facture : l'argent encaissé par l'hôtel qui appartient au restaurant.
`statut`	enum	`A_REVERSER` → `REVERSE` (« Marquer reversé »).
`reverseLe`	instant	Date de remise de l'argent.

8. Résumé : quelle règle est protégée où

Règle métier	Protégée par
Pas de surbooking	`EXCLUDE reservation_sans_chevauchement` (hebergement)
Pas de saisons qui se chevauchent	`EXCLUDE saison_sans_chevauchement`
Pas de surpaiement	`CHECK facture_pas_de_surpaiement`
Un seul lien Papi en attente par facture	index unique partiel `paiement_un_en_attente_par_facture`
Une transaction Papi comptée une fois	`paiement.referenceExterne` unique
Stock jamais négatif	`CHECK stock_jamais_negatif` + décrément atomique
Numéros sans doublon	tables `compteur_*` (upsert + increment atomique)
Un événement traité une fois	`evenement_traite` (clé primaire) + uniques métier (`(origine, sourceId)`, `(referenceId, articleId)`, `(reservationId, type, datePrevue)`)
Aucun événement perdu	`outbox_evenement` (même transaction)
Deux modifications simultanées	`version` (verrou optimiste)
Un mot de passe / jeton volé en base inutilisable	`motDePasseHash` (scrypt), `jetonHash`, `notificationTokenHash` (SHA-256)

9. Ce qui changera avec le SaaS (Phase 2)

- **Toutes** les tables métier recevront `hotelId` (sauf les tables propres au client global).
- Les uniques deviendront **par hôtel** : `(hotelId, numero)` pour les chambres, `(hotelId, code)` pour les catégories et les articles, compteurs `(hotelId, annee)`.
- `utilisateur` deviendra **global** ; une nouvelle table `membre (utilisateurId, hotelId, role)` portera les rôles du personnel.
- `outbox_evenement` et les événements porteront le `hotelId`.
- Les politiques **RLS** de PostgreSQL filtreront par `hotelId` (personnel) ou par `clientId` (client).